

← Back to Workshop

Day 1 — Foundations & RAG

Session 1

Session 2

Session 3

 Workshop Home > Day 1

Day 1 — Foundations of Generative AI & RAG

Set up your AI development environment, understand how Large Language Models work, build your first AI chatbot, master LangChain basics, and create a full RAG pipeline from scratch.

 9:00 AM – 5:00 PM

3 Sessions

3 Hands-on Projects

Python · Ollama · LangChain · ChromaDB

Student Setup Checklist (Complete Before Workshop Starts)

1. Python 3.10+ & Virtual Environment

Verify installation: `python3 --version`.
Create & activate venv:
`python3 -m venv venv && source venv/bin/activate`

2. Ollama & Required Models

Download from ollama.com.
Run in terminal:
`ollama pull llama3.2`
`ollama pull nomic-embed-text`

3. Low-RAM Fallback Options

For laptops with 4GB/8GB RAM, pull lighter models:
`ollama pull smollm:135m` or `tinyllama`

4. VS Code & Extensions

Install VS Code + **Python**, **Pylance**, and **Jupyter** extensions. Test terminal access.

 Download all workshop dependencies & starter files:

 [Download requirements.txt](#)

SESSION 1 OF 3

 9:00 – 11:00 AM

Introduction to Generative AI & LLMs

CORE CONCEPTS



Intuitive Analogy — LLM vs AI Agent

A standard Chatbot functions like an intelligent text generator that responds when prompted. In contrast, an

AI Agent

 acts as an autonomous digital worker equipped with tools, memory, and reasoning loops to accomplish multi-step objectives independently.



AI vs ML vs Deep Learning

Understanding the hierarchy of intelligence systems:

- › **AI** — any technique making machines smart (rules, logic)
- › **ML** — machines learn from data (linear regression, decision trees)
- › **Deep Learning** — neural networks with many layers
- › **Gen AI** — models that generate new content (text, images, code)



Evolution of LLMs

The rapid progression of language models:

- › **2017** — "Attention is All You Need" paper (Transformers)
- › **2018** — BERT by Google (bidirectional)
- › **2019** — GPT-2 by OpenAI
- › **2022** — ChatGPT launches (GPT-3.5)
- › **2023** — Llama, Mistral (open-source)
- › **2024** — Llama 3, GPT-4o, Claude 3



How Transformers Work

The core architecture powering all modern LLMs:

- › **Tokenization** — text split into tokens (words/subwords)
- › **Embeddings** — tokens converted to number vectors
- › **Attention** — model learns which tokens relate to each other
- › **Feed Forward** — dense layers process attention output
- › **Output** — predicts next token, repeat until done



Prompt Engineering

Techniques to get better responses from LLMs:

- › **Zero-shot** — Ask directly: "Translate to French: Hello"
- › **Few-shot** — Give examples before asking
- › **Chain-of-Thought** — "Think step by step..."
- › **System Prompt** — Define role/persona for the AI
- › **Temperature** — Controls randomness (0=deterministic)



APIs vs Local Models

FEATURE	API (OPENAI)	LOCAL (OLLAMA)
Cost	Pay per token	Free
Privacy	Data sent to cloud	100% private
Speed	Fast (GPUs)	Depends on hardware
Models	GPT-4, etc.	Llama, Mistral
Internet	Required	Not needed



What is Agentic AI?

An AI Agent = LLM + Tools + Memory + Planning

- › **LLM** — the brain (reasoning)
- › **Tools** — web search, calculator, APIs
- › **Memory** — remember past conversations
- › **Planning** — break tasks into steps
- › Example: "Research topic X and write a report" → agent searches, reads, writes



Key Insight for Interviews

The difference between a Chatbot and an AI Agent: A chatbot only responds. An agent can take actions — search the web, write files, run code, and make decisions autonomously.



AI Career Paths for CSE Graduates

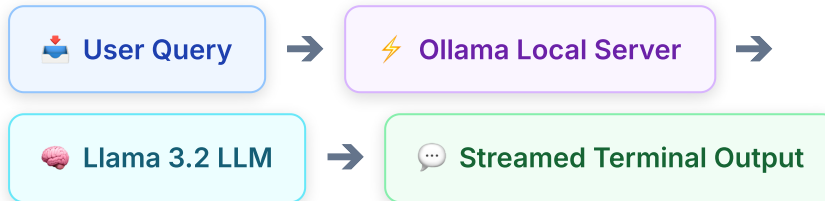
AI Engineer · ML Engineer · LLM Application Developer · Prompt Engineer · AI Product Manager · Data Scientist · AI Researcher



What you'll build

A Python environment running Ollama locally to invoke Llama 3.2, stream real-time tokens, and handle conversational turns.

PROGRAM ARCHITECTURE — LOCAL AI CHATBOT



What you'll build

A command-line AI chatbot that runs completely locally on your machine using Ollama + Llama 3.2. You type a message, the AI responds. No API key, no internet, no cost.

1

Install Python 3.11+

Download from python.org. During install, check ☒ "Add Python to PATH".
Verify:

BASH

Terminal

```
python --version
# Expected: Python 3.11.x or higher

pip --version
# Expected: pip 23.x ...
```

2

Install VS Code

Download from code.visualstudio.com. Install extensions: **Python**, **Pylance**.

3

Install Ollama (Local LLM Runner)

Go to ollama.com → Download for your OS → Install. Then pull Llama 3.2:

BASH

Terminal

```
ollama pull llama3.2
# Downloads ~2GB model – wait for completion

ollama run llama3.2
# Test it: type "Hello!" and press Enter
# Type /bye to exit
```



Low RAM?

Use `ollama pull llama3.2:1b` instead (1B parameter model, uses ~800MB RAM)

4

Create project folder and install dependencies

BASH

Terminal

```
mkdir agentic-ai-workshop
cd agentic-ai-workshop
pip install requests langchain langchain-ollama langchain-comm
```

5

Build your first AI Chatbot

Create a file called **chatbot.py** and paste this code:

```
# Day 1 - Session 1: Your First AI Chatbot

import requests
import json

# Ollama runs locally on port 11434
OLLAMA_URL = "http://localhost:11434/api/generate"
MODEL = "llama3.2"

def ask_ai(message, history=[]):
    """Send a message to Llama 3.2 and get a response"""

    # Build conversation context
    context = ""
    for msg in history[-4:]: # last 4 exchanges
        context += f"User: {msg['user']}\nAssistant: {msg['ai']}"

    full_prompt = f"""{context}User: {message}\nAssistant: """

    response = requests.post(OLLAMA_URL, json={
        "model": MODEL,
        "prompt": full_prompt,
        "stream": False,
        "options": {"temperature": 0.7, "num_predict": 500}
    })

    return response.json()["response"]

def main():
    print("👑 AI Chatbot powered by Llama 3.2 (type 'quit' to quit)")
    print("-" * 50)

    history = []

    while True:
        user_input = input("\nYou: ").strip()

        if user_input.lower() in ['quit', 'exit', 'bye']:
            print("Goodbye! 🙌")
            break

        if not user_input:
            continue

        print("AI: Thinking...", end="\r")
        response = ask_ai(user_input, history)
        print(f"AI: {response}")
```

```
history.append({"user": user_input, "ai": response})

if __name__ == "__main__":
    main()
```

6

Run and test!

BASH Terminal

```
python chatbot.py
```

✓ EXPECTED OUTPUT

🤖 AI Chatbot powered by Llama 3.2 (type 'quit' to exit)

You: What is machine learning?

AI: Machine learning is a subset of artificial intelligence where systems learn from data to improve performance on tasks without being explicitly programmed...

📄 Download Session 1 Python Code:

⬇️ [Download day1_01_first_chatbot.py](#)

🔥 COMMON ISSUES & TROUBLESHOOTING

Error: Connection Refused / Cannot connect to Ollama

Fix: Open a new terminal window and run `ollama serve` to ensure the background daemon is running.

Error: model 'llama3.2' not found

Fix: Run `ollama pull llama3.2` in your terminal before launching your Python script.

SESSION 1 CHECKPOINT & EXPECTED DELIVERABLES

- ✅ Python 3.10+ virtual environment active with `langchain-ollama` installed.
- ✅ Ollama running locally on port 11434 with `llama3.2` pulled.
- ✅ Working `day1_01_first_chatbot.py` streaming tokens in real-time.

Session 2: LangChain →

SESSION 2 OF 3

 11:15 AM – 1:00 PM

LangChain Fundamentals

CORE CONCEPTS



Intuitive Analogy — LangChain Framework

Think of LangChain as a modular

AI design system

. Instead of writing complex custom code, you connect pre-built components—Prompts, LLM Models, Tools, and Output Parsers—into robust end-to-end applications.



What is LangChain?

LangChain is a Python framework for building LLM-powered applications. It connects LLMs with prompts, memory, tools, and data sources through a unified interface — making it the #1 most used AI framework in production.



LangChain Architecture

Six core building blocks:

- › **Models** — LLM/Chat model wrappers
- › **Prompts** — PromptTemplate, ChatPromptTemplate
- › **Chains** — Connect components with | pipe
- › **Memory** — Store conversation history
- › **Tools** — External capabilities (search, calc)
- › **Agents** — LLM decides which tool to use



Prompt Templates

Templates let you create reusable prompts with dynamic variables:

- › PromptTemplate — simple string templates
- › ChatPromptTemplate — for chat models (system + human)
- › Variables use {curly_braces} syntax
- › Can include few-shot examples



LCEL — Pipe Syntax

LangChain Expression Language uses | to chain components:

- › prompt | llm | parser
- › Each component's output → next component's input
- › Easy to read, debug, and modify
- › Supports parallel execution



Output Parsers

Parse LLM text output into structured formats:

- › StrOutputParser — returns clean string
- › JsonOutputParser — parses JSON response
- › PydanticOutputParser — validates with data model
- › CommaSeparatedListOutputParser



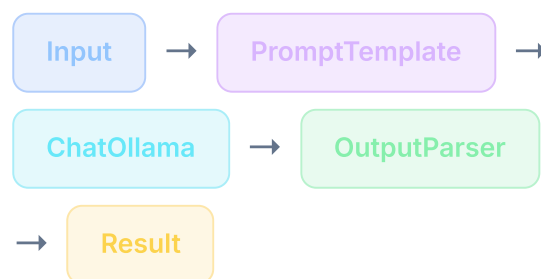
Memory

Enable conversations by remembering previous messages:

- › ConversationBufferMemory — stores all messages
- › ConversationBufferWindowMemory — last N messages
- › ConversationSummaryMemory — LLM summarizes history



LangChain Pipeline Flow



All connected using LCEL pipe | syntax



AI Resume Reviewer

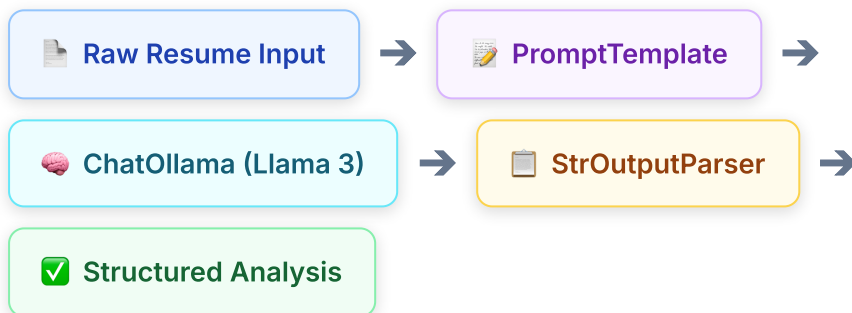
Paste a resume → get detailed strengths, weaknesses, and suggestions



What you'll build

An AI-powered resume analyzer using LangChain + Llama 3.2 that reads a resume and gives structured feedback: top 3 strengths, 3 weaknesses, and actionable improvement suggestions with a score out of 10.

PROGRAM ARCHITECTURE — LANGCHAIN LCEL CHAIN



PYTHON resume_reviewer.py

```

# Day 1 - Session 2: AI Resume Reviewer
from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1. Initialize the model
llm = ChatOllama(model="llama3.2", temperature=0.3)

# 2. Create the prompt template
prompt = ChatPromptTemplate.from_messages([
    ("system", """"You are an expert HR recruiter and career coach with 10+ years of experience. Analyze the given resume and provide structured, honest, and actionable feedback."""),
    ("human", """"Please review this resume and provide:

    **SCORE:** Give an overall score out of 10

    **STRENGTHS** (top 3):
    - Point 1
    - Point 2
    - Point 3

    **AREAS TO IMPROVE** (top 3):
    - Point 1
    - Point 2
    - Point 3

    **ACTION ITEMS** (specific suggestions):
    - Suggestion 1
    - Suggestion 2
    - Suggestion 3

    **INTERVIEW READINESS:** One sentence summary

    Resume:
    {resume}
    """)
])

# 3. Build the chain using LCEL (pipe syntax)
parser = StrOutputParser()
chain = prompt | llm | parser

# 4. Sample resume to test with

```

```

sample_resume = """
Name: Rahul Kumar
Email: rahul@email.com | LinkedIn: linkedin.com/in/rahulkumar

EDUCATION
B.E. Computer Science | Jyothy Institute of Technology | 2022-2024
CGPA: 7.8/10

SKILLS
Programming: Python, Java, C++
Web: HTML, CSS, JavaScript (basic)
Tools: Git, VS Code

PROJECTS
Library Management System - Built using Python and MySQL
Simple Calculator App - Android app using Java

INTERNSHIP
Web Development Intern | ABC Company | June 2024 (1 month)
- Worked on HTML/CSS pages
- Attended team meetings

ACHIEVEMENTS
- Participated in college hackathon
- Member of coding club
"""

# 5. Run the chain
print("🔍 Analyzing resume... Please wait\n")
result = chain.invoke({"resume": sample_resume})
print(result)
print("\n" + "="*50)
print("✅ Review complete! Replace sample_resume with your own")

```

2

Run it

BASH

```
python resume_reviewer.py
```

3

Challenge: Try with your own resume!

Replace sample_resume with your actual resume content and see what the AI suggests.



AI Email Generator

Generate professional emails in different tones with one command

PYTHON

email_generator.py

```

# Day 1 - Session 2: AI Email Generator
from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

llm = ChatOllama(model="llama3.2", temperature=0.7)

prompt = ChatPromptTemplate.from_messages([
    ("system", """You are a professional email writer.
    Write clear, concise emails appropriate for the given tone
    Format with Subject, greeting, body, and sign-off."""),
    ("human", """
    Write an email with the following details:
    - Tone: {tone}
    - From: {sender}
    - To: {recipient}
    - Purpose: {purpose}
    - Key points to include: {key_points}
    """)
])

chain = prompt | llm | StrOutputParser()

# Interactive email generator
print("✉ AI Email Generator")
print("=" * 40)

tone = input("Tone (formal/casual/persuasive/apologetic): ")
sender = input("Your name: ")
recipient = input("Recipient (e.g., 'HR Manager at Google'): ")
purpose = input("Purpose of email: ")
key_points = input("Key points to include (comma separated): ")

print("\n✉ Generating email...\n")
print("=" * 40)

result = chain.invoke({
    "tone": tone,
    "sender": sender,
    "recipient": recipient,
    "purpose": purpose,
    "key_points": key_points
})

```

```
print(result)
```

✅ EXAMPLE INTERACTION

Tone: formal

Your name: Basavesh

Recipient: HR Manager at Infosys

Purpose: Follow up on internship application

Key points: applied 2 weeks ago, strong Python skills, eager to contribute

 Download Session 2 Python Code:

 [Download day1_02_resume_reviewer.py](#)

🚨 COMMON ISSUES & TROUBLESHOOTING

Deprecation Warning: LangChain import error

Fix: Ensure you import from `langchain_ollama` and `langchain_core` (LangChain 0.3+ syntax).

🎯 SESSION 2 CHECKPOINT & EXPECTED DELIVERABLES

- ✅ Built modular LCEL chain using `PromptTemplate` | `ChatOllama` | `StrOutputParser`.
- ✅ Structured ATS score and section rewrites generated from raw resume text input.

[← Session 1](#)

[Session 3: RAG →](#)

Retrieval Augmented Generation (RAG)

CORE CONCEPTS



Intuitive Analogy — RAG & Vector Search

A

Vector Database

indexes data by *semantic meaning* rather than keyword matches.

RAG

provides the LLM with an *open-book retrieval system*—searching your documents for authoritative context before generating factual answers.



The Problem RAG Solves

LLMs have a training cutoff — they don't know about your college handbook, recent news, or your private documents. Asking GPT about your university's exam policy? It will hallucinate. RAG fixes this by letting the model "read" your documents first.



What are Embeddings?

Embeddings convert text into numerical vectors (lists of numbers) that capture meaning:

- › "King" and "Queen" have similar vectors
- › "Cat" and "Dog" are closer than "Cat" and "Car"
- › Similarity is measured by cosine distance
- › We use `nomic-embed-text` model via Ollama



Vector Databases

Specialized databases that store and search embeddings:

- › **ChromaDB** — local, no setup required 
- › **Pinecone** — cloud-based, scalable
- › **FAISS** — Facebook's fast similarity search
- › Search finds similar vectors (not exact keyword match)



Text Splitting

Large documents must be split into chunks before embedding:

- › `chunk_size=500` — characters per chunk
- › `chunk_overlap=50` — overlap prevents cutting mid-sentence
- › Too large → loses precision
- › Too small → loses context



Full RAG Pipeline

 Load Documents



 Split into Chunks



 Create Embeddings



 Store in ChromaDB



 User Asks Question



 Retrieve Relevant Chunks



 LLM Generates Answer

⚡ HANDS-ON PROJECT: COLLEGE QUESTION ANSWERING SYSTEM



College Q&A System

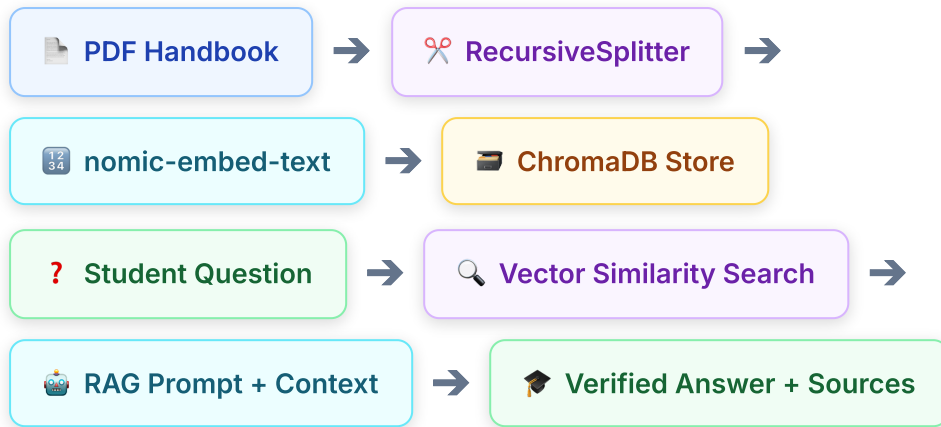
Load any PDF → Ask questions → Get answers with source citations



What you'll build

A RAG pipeline that loads your college handbook or any PDF, creates a vector database, and answers questions with exact source references. This is the foundation of every enterprise AI assistant.

PROGRAM ARCHITECTURE — RAG VECTOR QA PIPELINE



1

Install additional dependencies

BASH

```
pip install chromadb pypdf langchain-chroma

# Pull the embedding model
ollama pull nomic-embed-text
```

PYTHON college_qa.py

```

# Day 1 - Session 3: College Question Answering System (RAG)
import os
from langchain_community.document_loaders import PyPDFLoader,
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_chroma import Chroma
from langchain_ollama import OllamaEmbeddings, ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

# — Configuration —
MODEL = "llama3.2"
EMBED_MODEL = "nomic-embed-text"
CHROMA_PATH = "./chroma_db"
CHUNK_SIZE = 500
CHUNK_OVERLAP = 50

def load_documents(file_path: str):
    """Load PDF or text file"""
    print(f"📄 Loading document: {file_path}")

    if file_path.endswith('.pdf'):
        loader = PyPDFLoader(file_path)
    else:
        loader = TextLoader(file_path)

    docs = loader.load()
    print(f"✅ Loaded {len(docs)} page(s)")
    return docs

def split_documents(docs):
    """Split documents into smaller chunks"""
    print(f"✂️ Splitting into chunks...")
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=CHUNK_SIZE,
        chunk_overlap=CHUNK_OVERLAP,
        separators=["\n\n", "\n", " ", ""]
    )
    chunks = splitter.split_documents(docs)
    print(f"✅ Created {len(chunks)} chunks")
    return chunks

def create_vector_db(chunks):
    """Create and persist ChromaDB vector store"""
    print(f"🗄️ Creating embeddings and storing in ChromaDB...")

```

```

embeddings = OllamaEmbeddings(model=EMBED_MODEL)
vectordb = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory=CHROMA_PATH
)
print("✅ Vector database created!")
return vectordb

def load_vector_db():
    """Load existing ChromaDB"""
    embeddings = OllamaEmbeddings(model=EMBED_MODEL)
    return Chroma(
        persist_directory=CHROMA_PATH,
        embedding_function=embeddings
    )

def build_rag_chain(vectordb):
    """Build the RAG question-answering chain"""
    llm = ChatOllama(model=MODEL, temperature=0)
    retriever = vectordb.as_retriever(
        search_type="similarity",
        search_kwargs={"k": 3} # retrieve top 3 chunks
    )

    prompt = ChatPromptTemplate.from_messages([
        ("system", """You are a helpful assistant for answering
        about college documents. Use ONLY the provided context.
        If the answer is not in the context, say "I don't have
        in the provided documents."

        Always mention which part of the document your answer
        ("human", """Context from documents:
        {context}

        Question: {question}

        Answer (with source reference):""")
    ])

    def format_docs(docs):
        return "\n\n".join([
            f"[Source: Page {doc.metadata.get('page', 'N/A')}]'"
            for doc in docs
        ])

    chain = (
        {"context": retriever | format_docs, "question": Runnable
        | prompt
        | llm

```

```

        | StrOutputParser()
    )
    return chain

def main():
    print("🎓 College Q&A System (RAG)")
    print("=" * 45)

    # Check if DB exists, else create it
    if not os.path.exists(CHROMA_PATH):
        pdf_path = input("Enter path to your PDF/TXT file: ").strip()
        if not os.path.exists(pdf_path):
            print("File not found! Creating demo with sample text")
            # Create sample text file for demo
            with open("college_info.txt", "w") as f:
                f.write("""
Jyothy Institute of Technology - College Handbook

EXAMINATION RULES:
Students must carry their hall ticket to all examinations.
Mobile phones are strictly prohibited in examination halls.
Minimum 75% attendance is required to appear for exams.
Students found cheating will be expelled from the examination.

ATTENDANCE POLICY:
Regular attendance is mandatory for all students.
Students with less than 75% attendance will be detained.
Medical leave requires a medical certificate from a registered doctor.
Prior permission must be obtained for planned absences.

FEE STRUCTURE:
Tuition fee: Rs. 80,000 per year
Hostel fee: Rs. 60,000 per year (including meals)
Lab fee: Rs. 5,000 per semester
Bus facility available for selected routes.

LIBRARY RULES:
Library hours: 8:00 AM to 8:00 PM on working days
Students can borrow up to 3 books at a time
Books must be returned within 14 days
Late fee: Rs. 2 per day per book

PLACEMENT CELL:
The Placement Cell coordinates campus recruitment.
Students with CGPA above 6.0 are eligible for placement.
Mock interviews and aptitude training provided.
Companies visiting include TCS, Infosys, Wipro, Accenture.
                """)
            pdf_path = "college_info.txt"

```

```

docs = load_documents(pdf_path)
chunks = split_documents(docs)
vectordb = create_vector_db(chunks)
else:
    print("📁 Loading existing database...")
    vectordb = load_vector_db()

chain = build_rag_chain(vectordb)

print("\n🤖 Ready! Ask questions about the college document")
print("Type 'quit' to exit.\n")

while True:
    question = input("? Your question: ").strip()
    if question.lower() in ['quit', 'exit']:
        break
    if not question:
        continue

    print("\n🤖 Answer:")
    answer = chain.invoke(question)
    print(answer)
    print("\n" + "-" * 45 + "\n")

if __name__ == "__main__":
    main()

```

3

Run the Q&A System

BASH

```

python college_qa.py

# Try these questions:
# "What is the attendance requirement?"
# "What are the exam rules?"
# "How much is the hostel fee?"
# "What companies visit for placement?"

```

✅ EXPECTED OUTPUT

? Your question: What is the minimum attendance required?

🤖 Answer: According to the college handbook (Attendance Policy section), the minimum attendance required is **75%**. Students with less than 75% attendance

will be detained and will not be allowed to appear for examinations. Medical absences require a certificate from a registered doctor...



Placement Insight ★★★★★

RAG is the most in-demand AI skill in 2024. Every company building an AI assistant over their internal documents uses RAG. Master this and you're immediately interview-ready for AI Engineer roles.



Download Session 3 Python Code:



Download day1_03_college_qa_rag.py



COMMON ISSUES & TROUBLESHOOTING

Error: ChromaDB sqlite3 / C++ Compilation Error

Fix: On Linux/macOS, install `pip install pysqlite3-binary` and add `__import__('pysqlite3'); sys.modules['sqlite3'] = sys.modules.pop('pysqlite3')` at the top of your script.

Error: model 'nomic-embed-text' not found

Fix: Run `ollama pull nomic-embed-text` in your terminal before running vector embeddings.



SESSION 3 CHECKPOINT & EXPECTED DELIVERABLES



Created persistent ChromaDB vector store in `./chroma_db`.



RAG pipeline retrieves relevant document chunks and returns answers with exact source citations.

← Session 2

Day 2: AI Agents →



